

Logical and stochastic modeling with SMART^{*}

G. Ciardo¹, R. L. Jones III², A. S. Miner³, and R. Siminiceanu¹

¹ Department of Computer Science, College of William and Mary

² ASRC Aerospace Corporation

³ Department of Computer Science, Iowa State University

Abstract. We describe the main features of SMART, a software package providing a seamless environment for the logic and probabilistic analysis of complex systems. SMART can combine different formalisms in the same modeling study. For the analysis of logical behavior, both explicit and symbolic state-space generation techniques, as well as symbolic CTL model-checking algorithms, are available. For the study of stochastic and timing behavior, both sparse-storage and Kronecker numerical solution approaches are available when the underlying process is a Markov chain. In addition, discrete-event simulation is always applicable regardless of the stochastic nature of the process, but certain classes of non-Markov models can still be solved numerically. Finally, since SMART targets both the classroom and realistic industrial settings as a learning, research, and application tool, it is written in a modular way that allows for easy integration of new formalisms and solution algorithms.

1 Introduction

Complex discrete-state systems such as computer and communication networks, distributed software, and factory assembly lines are increasingly engineered and placed in service in environments where their correct logical and timing behavior is essential. Thus, both their verification and their performance and reliability analysis are important tasks in the design and dimensioning of such systems. Logical verification is usually concerned with absence of design errors such as deadlocks and similar “untimed” and “non-stochastic” properties, while both performance and reliability analysis are usually concerned with timing behavior in a stochastic setting. The two are rarely considered at the same time.

Individually, both *model checking* tools such as NuSMV [14] and *performability* (i.e., combined performance and reliability) tools, such as UltraSAN [16] and Möbius [17] can be very useful to discover potential design flaws and bottlenecks in a system. Recently, however, there has been a clear trend toward combining these two aspects into a single unifying framework, as done in PRISM [22] (it is worth mentioning that there are also tools for verification of real-time systems, such as KRONOS [29] and UPPAAL [2]; however, while these tools combine logic and timing, they do not target probabilistic aspects). We believe that there are

^{*} This work was partially supported by the National Aeronautics and Space Administration under NASA Contract NAG-1-2168, NAG-1-02095, and NAS-1-99124; by the National Science Foundation under grants CCR-0219745 and ACI-0203971; by a joint STTR project with Genoa Software Systems, Inc., for the Army Research Office; and by the Virginia Center for Innovative Technology under grant FED-95-011.

two main reasons for this integration trend. First and foremost, in many systems, it might be necessary to discuss correctness in light of timing behavior (e.g., it must not be possible for events a and b to occur within x time units of each other), and to accept a probabilistic, rather than an absolute, view of correctness (e.g., the probability of non-termination due to entering a deadlock should be less than 10^{-9}). A second reason is that the *data structures and techniques* required to carry on either type of analysis have much in common, and can help each other. It is on this second aspect that we focus our presentation.

We introduce the tool SMART, whose development began in 1995. Designed as a powerful stochastic environment to integrate multiple modeling formalisms for use both in the classroom and in industrial applications, SMART has evolved to include logical analysis as well. Currently, it employs some of the most efficient data structures and algorithms known for the analysis of discrete-state systems.

2 Overview of SMART

At the heart of SMART lies the ability to define parametric *models* for which a variety of *measures* can be computed. A modeler may use multiple models to study different aspects of a system. Indeed, each model can be expressed in the most appropriate formalism, although, currently, the only high-level formalism is Petri nets [26], with a software-oriented formalism being planned, while the other available formalisms, discrete-time and continuous-time Markov chains (DTMCs and CTMCs), are rather low-level. Models can interact by exchanging data: a measure computed in a model can be an input parameter for another model.

For logical analysis, SMART can generate the (reachable) *state space* of a model using highly optimized algorithms ranging from *explicit* ones based on “flat” hashing or search trees or “structured” trees of trees [10], to *symbolic* ones based on multi-valued decision diagrams (MDDs) [21, 24]. A particularly innovative aspect in SMART is the symbolic encoding of the *transition relation* $\mathcal{N}$, which specifies which states can be reached from each state in one step. Unlike traditional symbolic approaches where a “double-height” MDD encodes the “from” and “to” states in the relation, SMART uses either a boolean sum of Kronecker products of (small) boolean matrices [3], or a *matrix diagram* [11, 23]. For mostly asynchronous systems, not only is this encoding enormously more efficient in terms of memory, but it also allows us to exploit the inherent *event locality* to greatly improve the runtimes as well [7, 8, 9].

For stochastic timing analysis, both numerical solutions and simulation are available. The former are feasible if the underlying process is a DTMC, CTMC, or semi-regenerative process satisfying certain conditions [19]. For DTMCs and CTMCs, an explicit solution approach requiring $O(\eta(\mathbf{P}))$ or $O(\eta(\mathbf{Q}))$ memory is available, where $\eta(\cdot)$ is the number of nonzero entries in the argument and $\mathbf{P}$ and $\mathbf{Q}$ are the transition probability or infinitesimal generator matrices, respectively.

In the semi-regenerative case, an *embedded process*, a DTMC, must be recognized and built. The expected holding times in each state and the state-to-state transition probabilities in this process are obtained by solving a (usually large) set of *subordinate processes*: currently, SMART can compute them if these subordinate processes are CTMCs, but, in the future, we plan to extend the approach to general cases, even to models where the solution of some of the subordinate

processes may require the use of discrete-event simulation. Recognizing the embedded, or regeneration, instants is a fundamental capability for this approach. In SMART, the same capability is now exploited to recognize such instants when *regenerative simulation* is being used as the overall solution algorithm.

Being able to classify the type of stochastic process underlying a given model is of course highly desirable [6]. If the distributions appearing in a model are all geometric or all exponential, it is immediate to conclude that the underlying process is a DTMC or a CTMC, respectively (the converse is not true). Semi-regenerative processes, however, are much harder to recognize *a priori*: if we allow for the possibility of unbounded models and general distributions, the question is indeed undecidable, since it is equivalent to deciding whether two events in the model can ever be enabled concurrently and asynchronously. Thus, in SMART we have focused instead on efficient *on-line* classification of the underlying stochastic process during its generation. If the model is not clearly Markov, SMART attempts to generate its embedded and subordinate processes in the hope that it falls in the class of semi-regenerative processes it can solve. Currently, SMART uses an explicit data structure to store the embedded state space it generates during this attempt. However, we plan to devise an approach that uses symbolic data structures analogous to the ones employed for logical analysis, except that the notion of *state* needs to be extended to incorporate *relative ordering information* for the enabled events.

In summary, the interface of SMART presents the user with a unified syntax to specify a wide variety of systems and ask questions of their models. Internally, the numerous logical, numerical, and simulation solution algorithms are tightly integrated, helping each other by sharing important data structures that are efficiently implemented, and, most importantly, are available regardless of the modeling formalism employed by the user.

3 SMART Language

SMART uses a strongly-typed computation-on-demand language with five types of basic statements and two compound statements, which can be arbitrarily nested:

Declaration statements are used to declare *functions* over some set of *arguments*. As a special case, the set of arguments for a function can be empty, thus the function is constant. In this sense, however, “constant” should not be confused with “non-random” (i.e., deterministic). To ensure strict type-checking, the *type* of the function and of its arguments must be defined.

Definition statements are used to declare functions in the same way declaration statements do, but they also specify how to compute their value.

Model statements are used to define models. Like functions, they have arguments but instead of returning a value, they specify a block with *declarations*, *specifications*, and *measures*, the latter being accessible outside the model.

Expression statements are used to print values, although function calls appearing in the expression may have side-effects, such as redirecting the output or displaying additional information.

Option statements are used to modify the behavior of SMART. For example, there are options to control the numerical solution algorithms (such as the precision or the maximum number of iterations), the verbosity level, etc. Option statements appear on a single line beginning with “#”.

Compound for statements define arrays or repeatedly evaluate parametric expressions. This is particularly useful for studies that explore how a result is affected by a change in the modeling assumptions, such as the rate of an event or the maximum size of a buffer.

Compound converge statements specify fixed-point iterations such as those employed to carry out approximate performance or reliability studies. They cannot appear within the declaration of a model.

The following basic predefined *types* are available in SMART:

```
bool: the values true or false.                bool c := 3 - 2 > 0;
int: integers (machine-dependent).                int i := -12;
bigint: arbitrary-size integers. bigint i := 12345678901234567890 * 2;
real: floating-point values (machine-dependent).  real x := sqrt(2.3);
string: character-array values.                  string s := "Monday";
```

In addition, composite types can be defined using the concepts of:

```
aggregate: analogous to the Pascal “record” or C “struct”.      p:t:3
set: collection of homogeneous objects.                        {1..8,10,25,50}
array: homogeneous objects indexed by set elements, see Sect. 3.2.
```

A type can be further modified by the following *natures*, which describe stochastic characteristics:

```
const: (the default) a non-stochastic quantity.
ph: a random variable with discrete or continuous phase-type distribution.
rand: a random variable with arbitrary distribution.
proc: a random variable that depends on the state of a model at a given time.
```

Finally, predefined *formalism* types can be used to define stochastic processes evolving over time, see Sect. 3.5.

3.1 Function declarations

Objects in SMART are functions, possibly recursive, that can be overloaded:

```
real pi := 3.14;                // an argument-less function
bool close(real a, real b) := abs(a-b) < 0.00001;
int pow(int b, int e) := cond(e==1,b,b*pow(b,e-1));
real pow(real b, int e) := cond(e==1,b,b*pow(b,e-1));
pow(5,3); pow(0.5,3);           // prints 125, integer, and 0.125, real
```

3.2 Arrays

Arrays are declared using a **for** statement; their dimensionality is determined by the enclosing iterators. Since the indices along each dimension belong to a finite set, we can define arrays with **real** indices. For example,

```
for (int i in {1..5}, real r in {1..i..0.5}) {
  real res[i][r] := MyModel(i,r).out1;
}
```

fills array **res** with the value of measure **out1** for the parametric model **MyModel**, when the first input parameter, **i**, ranges from one to five and the second one, **r**, ranges from one to the value of the first parameter, with a step of 1/2. Thus, note that **res** is not a “rectangular” array of values.

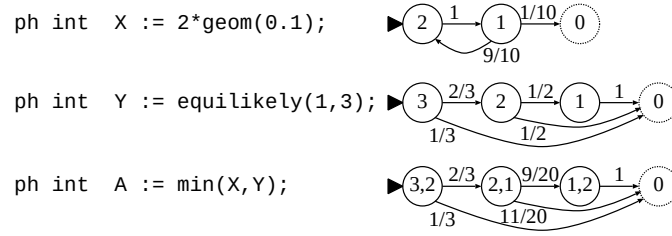


Fig. 1: The internal phase-type representation in SMART.

3.3 Fixed-point iterations

The approximate solution of a model is often based on a heuristic decomposition, where (sub)models are solved in a fixed-point iteration [13]. This can be specified with the `converge` statement:

```
converge {
  real x guess 1.0;           // initial guess for fixed-point iteration
  real y guess 2.0;
  real y := ModelA(x, y).measureY;
  real x := ModelB(x, y).measureX;
}
```

The `converge` statement iterates until the values of the variables declared in it differ by less than ϵ , in either relative or absolute terms, from one iteration to the next. Note that the values of these variables, `x` and `y` in our example, are updated either immediately or at the end of each iteration. The user can control both ϵ and the updating criterion using the appropriate option statements.

3.4 Random variables

SMART can manipulate discrete and continuous phase-type distributions of which the exponential (`expo`), geometric (`geom`), and non-negative integer constants are special cases. Combining `ph` types produces another `ph` type if phase-type distributions are closed under that operation:

```
ph int X := 2*geom(0.1);
ph int Y := equilikely(1,3); // Y = 1, 2, or 3 with probability 1/3
ph int A := min(X,Y);
ph int B := 3*X+Y;
ph int C := choose(0.4:X,0.6:4*Y);
ph real x := expo(3.2);
ph real y := erlang(4,5);
ph real a := min(3*x,y);
```

Internally, SMART uses an absorbing DTMC or CTMC to represent a `ph int` or `ph real`, respectively. These representations are actually built only if needed for a numerical computation. For example, if the expression `avg(A)` needs to be evaluated, SMART builds the internal representation of the DTMCs corresponding to `X`, `Y`, and `A` shown in Fig. 1, then computes the mean-time-to-absorption in the third DTMC, 1.96667. The number of states in the representation of a phase-type obtained through operators such as `max` and `min` can grow very rapidly.

Mixing `ph int` and `ph real` or performing operations not guaranteed to result in a phase-type distribution forces SMART to consider the resulting type as generally distributed (such general `rand` random variables can be manipulated only via Monte Carlo methods, currently under development):

```
rand int  D := X-Y;
rand int  F := X*Y;
rand real E := x+X;
```

3.5 Modeling formalisms

Components of a model are declared using formalism-specific types (e.g., the places of a Petri net). The model structure is specified by using formalism-specific functions (e.g., set the initial number of tokens in a place of a Petri net). Measures are user-defined functions that specify some constant quantity of interest (e.g., the expected number of tokens in a given place in steady-state), and are the only components accessible outside of the model definition block.

The design of SMART allows for relatively easy addition of new model formalisms. Currently, the `dtmc`, `ctmc`, and `spn` formalisms are implemented. For the `spn` formalism, the type of underlying stochastic process is determined by the distributions specified for the transitions. For example, the model shown on the right of Fig. 2 is defined by the `spn Net` on the left of the same figure, where `place` and `trans` are formalism-specific types; `arcs`, `firing`, and `init` are formalism-specific functions; measures such as `n.s` and `speed` are accessed outside the model using “.” notation. Since all firing time distributions are specified as `expo`, the underlying process is automatically recognized to be a CTMC. The statements after the model produce the output

```
n=1: 5 states, 8 arcs, throughput = 0.292547
n=2: 14 states, 34 arcs, throughput = 0.456948
n=3: 30 states, 88 arcs, throughput = 0.553456
n=4: 55 states, 180 arcs, throughput = 0.612828
n=1: E[tk(p5,p4,p3,p2,p1)] = (0.265952,0.469362,0.264686,0.208963,0.525085)
```

where the measure calls cause SMART to perform the appropriate analysis, steady-state solution of the underlying CTMC in this case. The five-state CTMC for `n=1` is shown at the bottom of the same figure: states are identified by the number of tokens in places p_5 , p_4 , p_3 , p_2 , and p_1 , in order, and arcs are labeled with the rate of the transition responsible for each marking change. For example, the steady-state probability that p_5 contains a token is the same as the probability of occupying state (10000) in the CTMC, 0.265952; multiplying it by the rate of transition a gives 0.2925472, the throughput of a , as shown in the `print` output.

4 Advanced features

In addition to the features already illustrated in the previous section, SMART possesses several more that make it truly unique and powerful, while relieving the user from having to know about details of the internal solution algorithms.

One important feature of model definition is the ability to define arrays of objects in a model. For example, consider a local-area network consisting of n workstations and one bus-like communication network, where each workstation i executes locally for an average amount of time $1/\lambda$, then it attempts to access

```

spn Net(int n) := {
  place p5, p4, p3, p2, p1;
  trans a, b, c, d, e;
  arcs(p5:a,a:p4,a:p2,p4:c,c:p3,p3:b,
       b:p4,p2:d,d:p1,p1:e,p3:e,e:p5);
  firing(a:expo(1.1),b:expo(1.2),
        c:expo(1.3),d:expo(1.4),e:expo(1.5));
  init(p5:n);
  bigint n_s := num_states(false);
  bigint n_a := num_arcs(false);
  real speed := avg_ss(rate(a));
  real e1 := avg_ss(tk(p1));
  real e2 := avg_ss(tk(p2));
  real e3 := avg_ss(tk(p3));
  real e4 := avg_ss(tk(p4));
  real e5 := avg_ss(tk(p5));
};

for (int n in {1..4}) {
  print("n=",n," : ",Net(n).n_s:2," states, ",
        Net(n).n_a:3," arcs, throughput = ",Net(n).speed,"\n");
}

print("n=1: E[tk(p5,p4,p3,p2,p1)] = (", Net(1).e5," ",
      Net(1).e4," ", Net(1).e3," ", Net(1).e2," ", Net(1).e1,")\n");

```

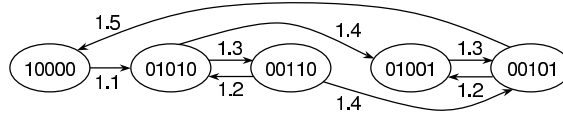
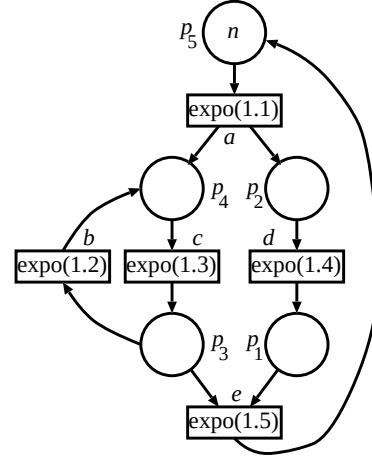


Fig. 2: SMART input file for the Petri net on the right, and underlying CTMC (for $n=1$).

the network. If the network is not in use, workstation i is granted access and sends data on the network for an average amount of time $1/\mu$. Otherwise, if some other workstation j is already using the network, i goes into a “backoff” state for an average amount of time $1/\delta$, then it attempts again to access the network. If the network is again busy (either because j is still using it, or because some other workstation k has gained access to it in the meantime), i waits for another backoff period, and so on, until it is finally granted access to it.

Assuming that all times are exponentially distributed, we can model this system as a CTMC. Since n is unknown, our model is parametric in the number of CTMC states; this is achieved by declaring arrays of places, as shown in Fig. 3, where we compute the steady-state expectation of the number of workstations in the backoff state and the probability that no workstation is in the backoff state for $\mu = 1$, $\delta = 0.1$, $\lambda = 0.04$, and $n = 2, 3, \dots, 20$.

4.1 Efficiency considerations

One defining aspect of the SMART language is its computation-on-demand operational semantics. An input file is parsed and type-checked as it is read, but no computation takes places until it is required to perform an output. Thus, for example, an `spn` model M may define transient measures `tm1` and `tm2` and

```

ctmc Lan(int n, real lambda, real mu, real delta) := {
  for (int i in {0..n-1}) {
    state idle[i], busy[i]; // "i" counts the workstations in backoff
  }
  init (idle[0]:1.0);
  for (int i in {0..n-1}) {
    arcs(idle[i]:busy[i):(n-i)*lambda, busy[i]:idle[i]:mu);
  }
  for (int i in {1..n-1}) {
    arcs(idle[i]:busy[i-1]:i*delta, busy[i-1]:busy[i):(n-i)*lambda);
    real aux[i] := prob_ss(in_state(idle[i])|in_state(busy[i]));
    real a[i] := cond(i>1,a[i-1]+i*aux[i],aux[1]);
  }
  real p := prob_ss(in_state(idle[0])|in_state(busy[0]));
};
real mu := 1.0; real delta := 0.1; real lambda := 0.04;
for (int n in {2..20}) {
  print("Prob[no backoff|n=",n,"]= ",Lan(n,lambda,mu,delta).p,"\\n");
  print("Avg[backoff|n=",n,"]= ",Lan(n,lambda,mu,delta).a[n-1],"\\n");
}

```

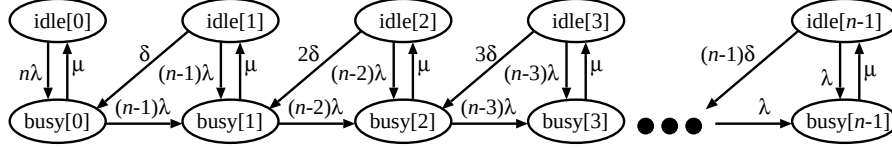


Fig. 3: A CTMC model with a parametric number of states.

steady-state measures **sm1** and **sm2**, but none of them will be computed when the model is parsed. An expression statement “**M.sm1**;” causes the **spn** to be solved in steady-state and the value of **M.sm1** to be computed and printed. However, a subsequent request to print **M.sm2** should not require the steady-state solution of **M** again. Thus, **SMART** employs a set of heuristics to decide appropriate *grouping* of measures. In our example, asking for either **M.sm1** or **M.sm2** would cause the computation of both, but not that of the transient quantities **tm1** and **tm2**.

A similar issue arises when a model is solved repeatedly with different parameters. For example, the **spn Net** in Fig. 2 requires a new generation of the state-space and of the underlying CTMC if repeatedly called with a different value of the parameter **n**. However, if it also had a parameter **lambda** for the rate of transition **a**, only the underlying CTMC, but not the state space, would have to be recomputed. Thus, **SMART** remembers the last set of parameters and measures a model has been solved for, and recomputes only what is needed when the same model is exercised again.

4.2 State-space generation and storage

The generation and storage of the state space of a model is a key component of any state-space-based solution technique, and it is an integral part of model checking. **SMART** implements a wide variety of techniques for constructing state spaces. There are *explicit* techniques that store each state individually and *im-*

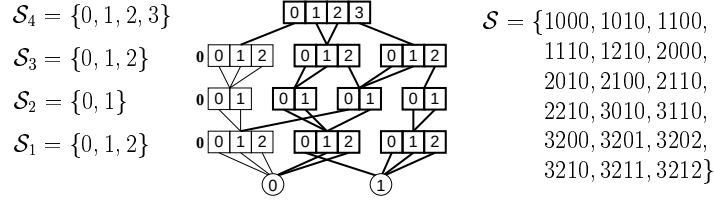


Fig. 4: An MDD example for a system partitioned into $K = 4$ components.

plicit techniques that employ multi-valued decision diagrams (MDDs) to symbolically store sets of states. This is governed by the choice of the `#StateStorage` option. Explicit algorithms, encompassing AVL and splay trees, and hash tables (option values `AVL`, `SPLAY`, `HASHING`), impose no restrictions on the model to be applicable, but the algorithms require time and memory at least linear in the number of reachable states. Symbolic algorithms, instead, require a *partition* of the model to exploit the system structure, but are normally much more efficient.

A partition of the model into K submodels implies that its (global) state can be written as the concatenation of K (local) states. In particular, the partition is *Kronecker-consistent* if the global model behavior can be expressed as a functional product of local behaviors for each submodel. For example, from a logical point of view, an event in the model is (globally) enabled iff it is (locally) enabled in each of the K submodels while, from a stochastic point of view, its (global) rate must be obtainable by multiplying some reference rate with the product of K dimensionless values, the k^{th} one depending only on the state of the k^{th} submodel. SMART automatically checks whether a partition specified by the user is Kronecker-consistent prior to attempting an analysis method that requires it (this check can be performed quite efficiently).

For Petri nets, a partition can be specified by directly assigning class indices (contiguous, strictly positive numbers) to places:

```
partition(2:p); partition(1:r); partition(1:t, 2:q, 1:s);
```

or by simply enumerating (without index information) the places in each class

```
partition(p:q, r:s:t);
```

In the latter case, the class indices are assigned automatically, in decreasing parsing order, thus the two examples shown have the same effect.

A common characteristic of symbolic approaches in SMART is the use of MDDs to encode sets of states. MDDs are a natural extension of the classic binary decision diagrams [4] to boolean functions of K multi-valued variables, $f : \{0, \dots, n_K - 1\} \times \dots \times \{0, \dots, n_1 - 1\} \rightarrow \{0, 1\}$. In particular we employ a *quasi-reduced* version of MDDs, where arcs can only connect nodes in adjacent levels. Given a partition of a model into K submodels, the local state space of the k^{th} submodel, $\mathcal{S}_k$, can be generated with explicit methods. The resulting local states are indexed $\{0, \dots, n_k - 1\}$, for a finite value of $n_k = |\mathcal{S}_k|$, hence a set of global states can be encoded as an MDD over the domain $\mathcal{S}_K \times \dots \times \mathcal{S}_1 = \hat{\mathcal{S}}$, called the *potential* state-space. An example of an MDD encoding is shown in Fig. 4 (nodes labeled with 0 encode the empty set and are not explicitly stored).

MDD-based algorithms are further distinguished by the iteration strategy employed for state-space exploration. One can choose a variant of breadth-

first search, `MDD_LOCAL_PREGEN`, locality-driven heuristics, `MDD_FORWARD_PREGEN` and `MDD_UPSTREAM_PREGEN` [7], or a *saturation* approach, `MDD_SATURATION` and `MDD_SATURATION_PREGEN` [8]. A unique feature of all symbolic approaches in SMART is the use of a Kronecker encoding of the transition relation between states. Unlike the BDD encoding of traditional symbolic techniques for binary relations, our Kronecker representation facilitates the detection and exploitation of *event locality*, a property inherently present in all asynchronous systems.

The most efficient iteration strategy in SMART is saturation, which exhaustively fires, in an MDD node at level k , all enabled events that affect only levels k and below, until the set encoded by the node reaches a fixed point. Subsequently, all the nodes below are also saturated, which means that the sub-MDD needs not be explored again, resulting in enormous reductions in computation time. Since only saturated nodes are stored, there is also a reduction in the peak storage requirements: often, the peak and final numbers of nodes are almost the same. This improves upon traditional approaches, in which the number of nodes can explode during generation, before “contracting” to the final representation.

There are two types of symbolic state-space generation approaches in SMART: the “`_PREGEN`” ones, which explicitly generate the local state spaces in isolation *a priori*, and the rest, which intertwine (explicit) local and (symbolic) global state-space generation. The former are slightly more efficient, but can be applied only when the submodels are “well-behaved” in isolation. The latter, instead, free the user from worrying about the behavior of submodels in isolation, and can indeed be applied whenever the overall state-space is finite [9].

Several predefined functions in SMART can be used to define measures that refer to the state space. For example, `num_states` returns the size of the state space and `num_arcs` returns the number of arcs in the reachability graph (both, optionally, can print the states, or the arcs, as a side-effect), while `reachable` returns the state space itself, for further manipulation (see Section 4.3).

Compared to other decision-diagrams-based tools, SMART excels at state-space generation for globally-asynchronous locally-synchronous systems, such as parallel and distributed protocols, where event locality is quite common. Fig. 5 shows the SMART input file describing the dining philosophers protocol (with deadlock) and a table reporting the runtime and memory consumption for state-space generation. The model can be scaled up to a huge number of philosophers, while requiring under four megabytes of memory and six minutes of runtime.

4.3 CTL model checking

Model checking is concerned with verifying temporal logic properties of discrete-state systems evolving in time. SMART implements the branching time *Computation Tree Logic* (CTL), widely used in practice due to its simple yet expressive syntax [15]. In CTL, operators occur in pairs: the path quantifier, either A (on all future paths) or E (there exists a path), is followed by the tense operator, one of X (next), F (future, finally), G (globally, generally), and U (until).

CTL model-checking queries are available in SMART via a set of model-dependent measures with type `stateset`. Each `stateset`, a set of states satisfying a given CTL formula, is stored as an MDD, and all MDDs for a model instance are stored in one *MDD forest*, sharing common nodes for efficiency. All

```

spn phils(int N) := {
  for (int i in {0..N-1}) {
    place
      idle[i], waitL[i], waitR[i], hasL[i], hasR[i], fork[i];
    partition(1+div(i,2):idle[i]:waitL[i]:waitR[i]:hasL[i]:hasR[i]:fork[i]);
    init(idle[i]:1, fork[i]:1);
    trans Go[i], GetL[i], GetR[i], Stop[i];
    firing(Go[i]:expo(1), GetL[i]:expo(1), GetR[i]:expo(1), Stop[i]:expo(1));
  }
  for (int i in {0..N-1}) {
    arcs(idle[i]:Go[i], Go[i]:waitL[i], Go[i]:waitR[i],
      waitL[i]:GetL[i], waitR[i]:GetR[i],
      fork[i]:GetL[i], fork[mod(i+1,N)]:GetR[i],
      GetL[i]:hasL[i], GetR[i]:hasR[i],
      hasL[i]:Stop[i], hasR[i]:Stop[i],
      Stop[i]:idle[i], Stop[i]:fork[i], Stop[i]:fork[mod(i+1, N)]);
  }
  bigint n_s := num_states(false);
};
# StateStorage MDD_SATURATION
print("The model has ", phils(read_int("N")).n_s, " states.\n");

```

Number of philosophers	States S	MDD Nodes		Mem. (bytes)		CPU (secs)
		Final	Peak	Final	Peak	
100	4.97×10^{62}	197	246	30,732	38,376	0.04
1,000	9.18×10^{626}	1,997	2,496	311,532	389,376	0.45
10,000	4.26×10^{6269}	19,997	24,496	3,119,532	3,821,376	314.13

Fig. 5: SMART code and computational requirements for the dining philosophers.

model-checking algorithms in SMART use symbolic techniques, and thus require a user-defined model partition. Four categories of model checking functions exist:

Atom builders: `nostates`, returns the empty set; `initialstate`, returns the initial state or states of the model; `reachable`, returns the set of reachable states in the model; `potential( $e$ )`, returns the states of $\hat{S}$ satisfying condition e ($\hat{S}$ depends on the model partition and the initial state).

Set operators: `union( $\mathcal{P}, \mathcal{Q}$ )`, returns $\mathcal{P} \cup \mathcal{Q}$; `intersection( $\mathcal{P}, \mathcal{Q}$ )`, returns $\mathcal{P} \cap \mathcal{Q}$; `complement( $\mathcal{P}$ )`, returns $\hat{S} \setminus \mathcal{P}$; `difference( $\mathcal{P}, \mathcal{Q}$ )`, returns $\mathcal{P} \setminus \mathcal{Q}$; `includes( $\mathcal{P}, \mathcal{Q}$ )`, returns `true` iff $\mathcal{P} \supseteq \mathcal{Q}$; `eq( $\mathcal{P}, \mathcal{Q}$ )`, returns `true` iff $\mathcal{P} = \mathcal{Q}$;

Temporal logic operators: the CTL operators `EX( $\mathcal{P}$ )`, `AX( $\mathcal{P}$ )`, `EF( $\mathcal{P}$ )`, `AF( $\mathcal{P}$ )`, `EG( $\mathcal{P}$ )`, `AG( $\mathcal{P}$ )`, `EU( $\mathcal{P}, \mathcal{Q}$ )`, `AU( $\mathcal{P}, \mathcal{Q}$ )`, and their dual counterparts in the past, `EXbar( $\mathcal{P}$ )`, `AXbar( $\mathcal{P}$ )`, `EFbar( $\mathcal{P}$ )`, `AFbar( $\mathcal{P}$ )`, `EGbar( $\mathcal{P}$ )`, `AGbar( $\mathcal{P}$ )`, `EUbar( $\mathcal{P}, \mathcal{Q}$ )`, `AUbar( $\mathcal{P}, \mathcal{Q}$ )`.

Execution trace output: `EFtrace( $\mathcal{R}, \mathcal{P}$ )`, `EGtrace( $\mathcal{R}, \mathcal{P}$ )` print a witness for `EF( $\mathcal{P}$ )`, `EG( $\mathcal{P}$ )` respectively, starting from a state in $\mathcal{R}$; `EUtrace( $\mathcal{R}, \mathcal{P}, \mathcal{Q}$ )`, prints a witness for `EU( $\mathcal{P}, \mathcal{Q}$ )` starting from a state in $\mathcal{R}$; `dist( $\mathcal{P}, \mathcal{Q}$ )`, returns the length of a shortest path from any state in $\mathcal{P}$ to any state in $\mathcal{Q}$.

Utility functions: `card( $\mathcal{P}$ )`, returns the number of states in $\mathcal{P}$ (as a `bigint`); `printset( $\mathcal{P}$ )`, prints the states in $\mathcal{P}$ (up to a given maximum).

The example in Fig. 6 shows a way to check for deadlocks in a generic model. The set D contains all reachable states that have no successors: if it is not empty,

```

spn MyModel := {
  ...
  stateset R := reachable;
  stateset S := EX(potential(true)); // states with a successor
  stateset D := difference(R,S);     // reachable deadlock states
  stateset P := potential(e_p);
  stateset T := EG(P);               // infinite run of X states
  bool    deadlock := neq(D,nostates);
  bool    dead     := printset(D);
  bool    trace    := EGtrace(initialstate,P);
}

```

Fig. 6: CTL queries on a generic model.

`MyModel.deadlock` is true, and evaluating the expression `MyModel.dead` causes all such deadlock states to be printed. It is also shown how to define a set of states `T`, from which there is an infinite path (i.e., a path with cycles) of states in `P`. Finally, the expression `MyModel.trace` can be used to generate a *witness* for $EG(\mathcal{P})$.

The counterexamples/witnesses feature in SMART allows the user to inspect execution traces for debugging purposes. The computation of traces in SMART is available for the fixed-point existential-type CTL operators: EF, EU, and EG. They are subject to several enhancing techniques that make SMART unique. The computation of EF traces can fully exploit the saturation algorithm, in conjunction with either a specific type of edge-valued decision diagrams (called EV^+ MDDs [12]), multi-terminal (or algebraic) decision diagrams, or simply forests of MDDs. These alternatives are controlled by the option `#TraceStorage`. An approach that only partially exploits saturation, is available instead for EU, while EG traces are implemented using traditional breadth-first search (hence they are not as efficient as the other two, but still benefit from the Kronecker representation of the transition relation and event locality).

4.4 Markov and non-Markov models

SMART provides standard numerical solutions to Markov models: power method and uniformization for transient analysis, iterative methods (Jacobi, Gauss-Seidel, SOR) for steady-state analysis. When all firing times are `ph int`, the underlying stochastic process is a DTMC, but on an expanded state space that encodes the distribution of the remaining firing time (the phase) of each enabled transition along with the structural state (the marking, for `spn` models). The resulting potential *phase space* is the *Kronecker product* of the state spaces of the absorbing DTMCs describing the firing time distributions. With `ph real` timing delays, the underlying process is a CTMC and the phase space is the *Kronecker sum* of the absorbing CTMCs describing the firing time distributions.

Mixing `ph int` and `ph real` timing delays in a model greatly complicates matters, but an `spn` model may still enjoy the Markov property if the phase advancements of the `ph int` transitions are *synchronized*. With the remaining firing time distributions already encoded in the state, the resulting stochastic process is semi-regenerative. As such, a single *embedded* DTMC arises separately from, but interacting with, many *subordinate* CTMCs corresponding to `ph real` firings. Reducing this otherwise difficult problem to Markov chain solutions allows

it to be tackled efficiently using standard numerical methods: **SMART** can compute steady-state measures of mixed **ph** models if it determines (automatically, while generating the state space) that the underlying process is semi-regenerative.

For example, consider the **spn** Net in Fig. 2 with $n=1$ but with firing delays `firing(a:expo(1.1), b:2, c:1, d:equilikely(1,2), e:expo(1.5));` so that both **ph real** and **ph int** firing delays are present. The phases of transition b are taken from $\{2, 1, 0, \bullet\}$, where “ $\bullet$ ” means that the transition is disabled. Upon enabling, b starts in phase 2, delays one unit of time, then moves to phase 1, where it delays again until phase 0, where it fires. Transition d behaves similarly over the same set of phases except it starts either in phase 1 or 2 with equal probability. The phase of **expo** transitions, one of $\{1, 0, \bullet\}$, does not need to be recorded: it is 1 if the transition is enabled, $\bullet$ if it is disabled, and 0 only for the instantaneous amount of time just before firing. Transitions b and c can be enabled concurrently and asynchronously with d . However, these transitions are “(phase-)synchronized”: c and d become simultaneously enabled when a fires and the alternation of the enablings of c and b preserves the phase synchronization with respect to d . This is a unique feature of our class of “synchronous **spn**” models, which have an underlying stochastic semi-regenerative process.

To solve a semi-regenerative process, we must first build its embedded DTMC. We observe the state of the model after each **ph int** phase change, if at least one such transition is enabled. Otherwise, we observe the state after each **ph real** phase change. Unlike **ph int** phase changes, which occur every unit of time, the time between **ph real** phase changes is an **expo** random variable that depends on the combined phase-change rate for all enabled **ph real** transitions, but it is unnecessary to explicitly record this time, thanks to the **expo** memoryless property. Fig. 7 shows the resulting embedded DTMC for **Net**(1) (with the firing time distributions just mentioned). States are identified by the number of tokens in p_5, p_4, p_3, p_2 , and p_1 , followed by the phase of transitions b, c , and d . The graph of the DTMC shows that **expo** transitions can fire from any state that enables them, while each enabled **ph int** transition must first go through a “phased delay”, until it reaches phase 0, when it immediately fires. The embedded state transitions require some explanation, however. State $(10000, \bullet \bullet \bullet)$, which enables transition a , can be exited in two ways, each with probability 0.5, since the firing of a enables transition d , which starts its phased delay from one of two phases with equal probability. The sojourn time of this embedded state has the same distribution as the firing delay of transition a : **expo**(1.1). Other state transitions have probability one except for those associated with the parameter α . These depend on whether the **expo** transition e fires before the **ph int** transition b moves out of phase 1 or phase 2 in discrete time. As **ph int** phase changes occur every time unit, we can compute α , the probability that transition e does not fire in the interval $[0, 1)$, by solving a two-state subordinate CTMC that starts in state 1 to indicate that transition e is in phase 1 while delaying an **expo**(1.5) time before transitioning to phase 0 and firing. Such a small CTMC has an easy solution: $\alpha = \exp(-1.5)$, or 0.77687. Since the embedded DTMC is irreducible and aperiodic, **SMART** computes a unique stationary probability vector, which, paired with the expected sojourn times in each state obtained from the subordinate CTMCs, is used to compute measures defined on the original semi-regenerative process. Our example produces the output

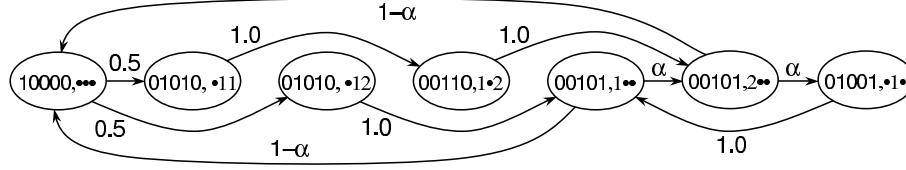


Fig. 7: Embedded DTMC underlying the synchronous **spn** derived from Fig. 2.

$n=1$: 7 states, 10 arcs, throughput = 0.31062

$n=1$: $E[\text{tk}(p_5, p_4, p_3, p_2, p_1)] = (0.282382, 0.355228, 0.36239, 0.46593, 0.251688)$

SMART employs also *embedding with elimination* [18, 20]: certain embedded states can be eliminated either when only **ph int** or only **ph real** transitions are enabled, or when both types are enabled but the **ph int** transitions require long phase evolutions before firing. In such cases, the embedded states become good candidates for elimination when their effect on the defined measures can just as easily be reformulated for inclusion within neighboring embedded states. In Fig. 7, state (00110, 1•2) serves no purpose other than imposing a **ph int** delay (no **ph real** delays and no transition firings). Thus, we can eliminate it and its transitions, bypass it with a new transition from (01010, •11) directly to (00101, 2••), still with probability one, set the sojourn time in (01010, •11) to 2 instead of 1, and, finally “distill” into state (01010, •11) any information about the eliminated state that is pertinent to the measures.

In the above synchronous **spn** example, we chose $n=1$ deliberately because, for $n>1$, the **spn** becomes *asynchronous*: **ph real** transition *a* can enable **ph int** transition *c* when **ph int** transition *d* is enabled, but in a way that causes their phases to be *unsynchronized*. Indeed, even with $n=1$, we can achieve asynchronous behavior by changing the firing delay of transition *b* to **ph real**. For asynchronous models, a separate clock is required in principle for each “group of synchronous phases”, but including these clocks within the state of the model results in a general state space Markov chain (GSSMC) with a continuous state space. SMART is able to recognize a class of asynchronous models that exhibit “regenerative regions” on these continuous state spaces. For these models, SMART implements a novel *regenerative simulation* method for steady-state analysis, based on identifying hidden regenerative regions within the underlying GSSMC. Identifying regeneration points requires that we fix a state (marking and phase) known to be visited infinitely often and observe visits to it that also have clock readings with a *distinguished probability distribution* fixed and independent of the past. Through a specialized data structure, the simulation algorithm is efficient and able to identify a broad variety of regenerative regions [20].

4.5 Kronecker encoding of the Markov chain matrix

For **spn** models where all transitions have **expo** distributions (or are immediate, i.e., can fire as soon as they are enabled), SMART provides advanced solution methods based on a Kronecker encoding of the transition rate matrix **R** of the underlying CTMC. These methods are quite effective in reducing the memory requirements and, just like MDD methods, they require a user-specified partition. Indeed, both MDD and Kronecker techniques can be employed for the

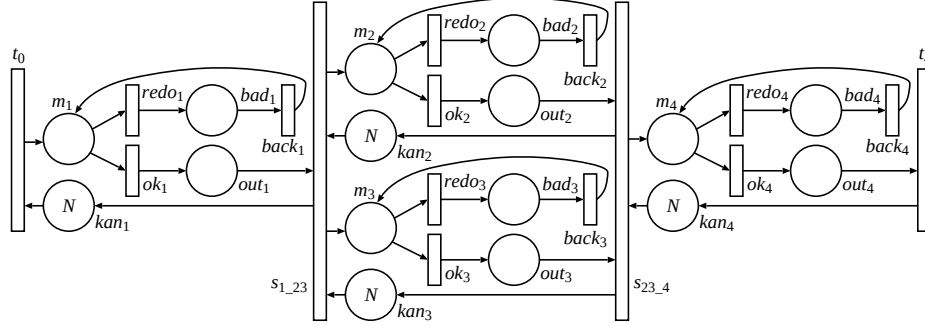
solution process, to store $\mathcal{S}$ and $\mathbf{R}$, respectively. This allows SMART to use probability vectors of size $|\mathcal{S}|$, which are then the only objects for which the memory requirements effectively limit the applicability of a numerical solution technique.

The technique used to store the transition rate matrix $\mathbf{R}$, and the multiplication algorithm used in the numerical iterations, are specified with the `#MarkovStorage` option. Explicit `SPARSE` storage is the most general, and the default option. Algorithms based on the potential state space $\hat{\mathcal{S}}$ or the actual state space $\mathcal{S}$ [5] for use with a Kronecker representation can be specified (`POTENTIAL_KRONECKER` and `KRONECKER`). Kronecker-based vector-matrix multiplication algorithms can be used with Jacobi by forcing SMART to store the representation “by rows” (set option `#MatrixByRows` to `true`); otherwise, SMART uses a Jacobi iteration that, like Gauss-Seidel, accesses the matrix by columns.

SMART also offers a particularly efficient data structure that combines the idea of decision diagrams with that of Kronecker algebra: *matrix diagrams* [11]. Selecting the option value `MATRIX_DIAGRAM_KRONECKER` causes SMART to transform the Kronecker representation into a matrix diagram, which has a significantly lower computational overhead during the numerical solution. For example, the model of Fig. 8 [28] describes an assembly line where “kanbans” are used to limit access to portions of the line, for a form of flow control. There are four assembly stations, and we accordingly use a partition into four submodels. The model is parameterized by the number of kanban tokens, N , initially assigned to each station, in p_1 , p_2 , p_3 , and p_4 . The figure also shows the SMART input file and a table reporting the time and memory requirements for a numerical solution. The number of states $|\mathcal{S}|$ and of nonzero entries $\eta(\mathbf{R})$ grow rapidly. However, with the Kronecker option, SMART can solve models having over one order of magnitude more states than with traditional sparse-storage approaches. Furthermore, from the columns showing the time required (seconds per iteration), we can conclude that the time overhead imposed by the Kronecker approach is smallest when using matrix diagrams. As discussed in [5], there is memory/time trade-off when employing a Kronecker encoding of the transition rate matrices: the Gauss-Seidel method uses less memory and converges in fewer iterations, but it requires more time per iteration because it must use an algorithm to multiply a vector by a single matrix column; the Jacobi method (in this case, using a relaxation parameter $\omega = 0.9$) can use a more efficient matrix-vector multiplication algorithm, but requires more iterations for convergence. Employing matrix diagrams, we attain the best of both worlds: both the number of iterations and the time per iterations are smaller than with any other Kronecker-based approach. Finally, the ordinary sparse storage implementation is quite efficient, but only as long as the entries of the transition rate matrix can all fit in main memory, thus it is not competitive for large models (a “—” indicates that the model could not be solved due to excessive memory requirements).

4.6 Approximations

The use of MDDs for $\mathcal{S}$ and a Kronecker encoding for $\mathbf{R}$ allow us to compactly represent enormous CTMCs. For numerical solution, the only remaining memory bottleneck is the solution vector. When $\mathcal{S}$ is too large, even just the solution vector alone requires excessive memory and other techniques, such as discrete-event simulation or approximations, must be used.



```

spn kanban(int N) := {
  for (int k in {1..4}) {
    place m[k], bad[k], kan[k], out[k];    init(kan[k]:N);
    partition(m[k]:bad[k]:kan[k]:out[k]);
    trans redo[k], ok[k], back[k];        firing(back[k]:expo(0.3));
    arcs(m[k]:redo[k], redo[k]:bad[k], m[k]:ok[k], ok[k]:out[k],
         bad[k]:back[k], back[k]:m[k]);
    real e[k] := avg_ss(tk(m[k])+tk(bad[k])+tk(out[k]));
  }
  trans t0, s1_23, s23_4, t4;
  firing(t0:expo(1.0), redo[1]:expo(0.36), ok[1]:expo(0.84), s1_23:expo(0.4),
        redo[2]:expo(0.42), ok[2]:expo(0.98), redo[3]:expo(0.39), ok[3]:expo(0.91),
        s23_4: expo(0.5), redo[4]:expo(0.33), ok[4]:expo(0.77), t4:expo(0.9));
  arcs(kan[1]:t0, t0:m[1], out[1]:s1_23, s1_23:kan[1], kan[2]:s1_23, s1_23:m[2],
        kan[3]:s1_23, s1_23:m[3], out[2]:s23_4, s23_4:kan[2], out[3]:s23_4,
        s23_4:kan[3], kan[4]:s23_4, s23_4:m[4], out[4]:t4, t4:kan[4]);
};
kanban(read_int("N")).e[4];

```

N	States	Nonzeros	Matrix diagram		Kronecker						Sparse	
	$ S $	$\eta(\mathbf{R})$	Mem. (bytes)	GS its sec/it	Mem. (bytes)	GS its sec/it	JOR its sec/it	GS its sec/it	JOR its sec/it	GS its sec/it	its	sec/it
5	2,546,432	24,460,016	21,667	139 73	9,486	214 148	527 74	214 19	—	—	—	—
6	11,261,376	115,708,992	32,702	185 336	14,106	289 723	713 359	—	—	—	—	—
7	41,644,800	450,455,040	46,678	238 1,290	20,388	374 2,923	—	—	—	—	—	—

Fig. 8: Kanban system.

Currently, SMART provides a novel approximation technique [25] for stationary analysis of models with an underlying structured CTMC which can still relay on complete knowledge of $\mathcal{S}$ and $\mathbf{R}$, thanks to their extremely compact MDD and matrix diagram encodings. The technique performs K approximate aggregations, where each aggregation is based on the structure of the MDD representing $\mathcal{S}$: states whose path in the MDD passes through the same downward pointer at level k are grouped together for aggregation k . The rate of transition between two groups of states can be determined efficiently from the Kronecker structure for $\mathbf{R}$; however, for some transitions, it is necessary to know the relative probability of a state within its group. Since the rates of each aggregated CTMC may depend on the probabilities computed for the other aggregations,

N	$ S $	Worst relative error		CPU (sec)
		Average number of tokens	Average firing rate	
5	2.55×10^6	+2.557%	-0.074%	0.84
6	1.13×10^7	+2.262%	-0.099%	1.38
7	4.16×10^7	+2.032%	-0.097%	2.19
30	4.99×10^{13}	—	—	462.48
66	1.99×10^{17}	—	—	13,424.50

Fig. 9: Accuracy and CPU time of the approximation for the Kanban system

fixed-point iterations are used to break cyclic dependencies. The results can be quite accurate. For example, Fig. 9 shows the worst relative error (where an exact comparison is possible) when computing the average number of tokens in each place and the average firing rate of each transition for the Kanban system of Fig. 8. The total CPU time required by the approximation is also shown.

5 Conclusions and future directions

We have presented the main features of SMART, a software tool for the logical and stochastic modeling of discrete-state systems. One of its goals is being able to solve truly large models, as this is needed to tackle practical systems. The data structures and solution algorithms implemented in SMART reflect years of research advancements, and are the means to make progress toward this goal. The SMART *User Manual*, examples, related publications, and instructions on how to obtain a copy are available at <http://www.cs.wm.edu/~ciardo/SMART/>.

Nevertheless, much work is still required, in both research and implementation. A tighter integration of the logical and stochastic analysis capabilities of SMART is desirable (e.g., CSL [1]). Research items on our “to-do list” have been mentioned throughout the paper. Implementation items include the spin-off of *libraries* for some of the most advanced functionalities, in order to make them available to the research community, and the creation of a *model database* which will contain both benchmark-type and realistic modeling examples, along with a detailed description, sample SMART input code, and resource requirements.

References

- [1] C. Baier, B. R. Haverkort, H. Hermanns, and J.-P. Katoen. Model checking continuous-time Markov chains by transient analysis. In *Proc CAV*, LNCS 1855, pp. 358–372, July 2000. Springer.
- [2] J. Bengtsson et al. New Generation of UPPAAL. In *Int. Workshop on Software Tools for Technology Transfer*, June 1998.
- [3] J. W. Brewer. Kronecker products and matrix calculus in system theory. *IEEE Trans. Circ. and Syst.*, CAS-25:772–781, Sept. 1978.
- [4] R. E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Trans. Comp.*, 35(8):677–691, Aug. 1986.
- [5] P. Buchholz, G. Ciardo, S. Donatelli, and P. Kemper. Complexity of memory-efficient Kronecker operations with applications to the solution of Markov models. *INFORMS J. Comp.*, 12(3):203–222, 2000.
- [6] G. Ciardo, R. German, and C. Lindemann. A characterization of the stochastic process underlying a stochastic Petri net. *IEEE TSE*, 20(7):506–515, July 1994.

- [7] G. Ciardo, G. Luetttgen, and R. Siminiceanu. Efficient symbolic state-space construction for asynchronous systems. *Proc. ICATPN*, LNCS 1825, pp. 103–122, June 2000. Springer-Verlag.
- [8] G. Ciardo, G. Luetttgen, and R. Siminiceanu. Saturation: An efficient iteration strategy for symbolic state space generation. *Proc. TACAS*, LNCS 2031, pp. 328–342, Apr. 2001. Springer-Verlag.
- [9] G. Ciardo, R. Marmorstein, and R. Siminiceanu. Saturation unbound. In *Proc. TACAS*, Apr. 2003. Springer-Verlag. To appear.
- [10] G. Ciardo and A. S. Miner. Storage alternatives for large structured state spaces. *Proc. Int. Conf. on Modeling Techniques and Tools for Computer Performance Evaluation*, LNCS 1245, pp. 44–57, June 1997. Springer-Verlag.
- [11] G. Ciardo and A. S. Miner. A data structure for the efficient Kronecker solution of GSPNs. *Proc. PNPM*, pp. 22–31, Sept. 1999. IEEE Comp. Soc. Press.
- [12] G. Ciardo and R. Siminiceanu. Using edge-valued decision diagrams for symbolic generation of shortest paths. *Proc. FMCAD*, LNCS 2517, pp. 256–273, Nov. 2002.
- [13] G. Ciardo and K. S. Trivedi. A decomposition approach for stochastic reward net models. *Perf. Eval.*, 18(1):37–59, 1993.
- [14] A. Cimatti, E. Clarke, F. Giunchiglia, and M. Roveri. NuSMV: a new Symbolic Model Verifier. *Proc. CAV*, LNCS 1633, pp. 495–499, July 1999. Springer.
- [15] E. M. Clarke and E. A. Emerson. Design and synthesis of synchronization skeletons using branching time temporal logic. *Proc. IBM Workshop on Logics of Programs*, pp. 52–71, 1981.
- [16] J. A. Couvillion et al. Performability modeling with UltraSAN. *IEEE Software*, pp. 69–80, Sept. 1991.
- [17] D. Daly, D. D. Deavours, J. M. Doyle, P. G. Webster and W. H. Sanders. Möbius: an extensible tool for performance and dependability modeling. LNCS 1786, pp. 332–336, 2000. Springer-Verlag.
- [18] R. L. Jones. Analysis of Phase-Type Stochastic Petri Nets with Discrete and Continuous Timing. Tech. Rep. CR-2000-210296, NASA Langley, June 2000.
- [19] R. L. Jones and G. Ciardo. On phased delay stochastic Petri nets: Definition and an application. *Proc. PNPM*, pp. 165–174, Sept. 2001. IEEE Comp. Soc. Press.
- [20] R. L. Jones III. Simulation and Numerical Solution of Stochastic Petri Nets with Discrete and Continuous Timing. Ph.D. thesis, College of William and Mary, Department of Computer Science, Williamsburg, VA, 2002.
- [21] T. Kam, T. Villa, R. Brayton, and A. Sangiovanni-Vincentelli. Multi-valued decision diagrams: theory and applications. *Multiple-Valued Logic*, 4(1–2):9–62, 1998.
- [22] M. Z. Kwiatkowska, G. Norman, and D. Parker. PRISM: Probabilistic Symbolic Model Checker. In *Proc. Comp. Perf. Eval. / TOOLS*, pp. 200–204, Apr. 2003.
- [23] A. S. Miner. Efficient state space generation of GSPNs using decision diagrams. In *Proc. DSN*, pp 637–646, Washington, DC, June 2002.
- [24] A. S. Miner and G. Ciardo. Efficient reachability set generation and storage using decision diagrams. *Proc. ICATPN*, LNCS 1639, pp. 6–25, June 1999. Springer.
- [25] A. S. Miner, G. Ciardo, and S. Donatelli. Using the exact state space of a Markov model to compute approximate stationary measures. *Proc. ACM SIGMETRICS*, pp. 207–216, June 2000. ACM Press.
- [26] T. Murata. Petri Nets: properties, analysis and applications. *Proc. of the IEEE*, 77(4):541–579, Apr. 1989.
- [27] E. Pastor, O. Roig, J. Cortadella, and R. Badia. Petri net analysis using boolean manipulation. *Proc. ICATPN*, LNCS 815, pp. 416–435, June 1994. Springer.
- [28] M. Tilgner, Y. Takahashi, and G. Ciardo. SNS 1.0: Synchronized Network Solver. In *Int. Workshop on Manufacturing and Petri Nets*, pp. 215–234, June 1996.
- [29] S. Yovine. Model checking timed automata. In *European Educational Forum: School on Embedded Systems*, pp. 114–152, 1996.